

Enterprise DevSecOps Project

Cosign + Syft + SBOM Enterprise Evidence Report

Image Signing, Signature Verification, SBOM Transparency, Provenance, SLSA Alignment, CycloneDX Mapping and Accepted-Risk Register

Prepared By	Jagriti Banerjee
Project Scope	Private DevSecOps lab - vulnerable Flask app, Docker image, GHCR image artifact, SBOM, Cosign signatures and attestations
Primary Artifact	ghcr.io/jazz00001/enterprise-devsecops:latest
Immutable Digest	sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764
Evidence Sources	Cosign verify output, Syft SPDX/CycloneDX SBOM, pip-audit report, Trivy image report, SLSA-style provenance predicate
Document Classification	Portfolio / Private Lab Security Evidence

Executive Positioning: This report turns the project supply-chain work into enterprise-grade evidence by showing immutable artifact identity, cryptographic verification, SBOM coverage, vulnerability context, remediation status, and accepted-risk decisions.

Note: The project is a controlled private lab. Some vulnerable dependencies were intentionally retained during testing to demonstrate detection, documentation, remediation workflow, and risk governance.

1. Executive Summary

The supply-chain security implementation for the Enterprise DevSecOps project uses Cosign for container image signing and verification, Syft for SBOM generation, SPDX and CycloneDX for machine-readable component inventories, and SLSA-style provenance for build traceability. This combination demonstrates how a container artifact can move from a simple Docker image to a verifiable enterprise artifact with identity, integrity, inventory, vulnerability context, and governance decisions.

The strongest enterprise evidence is the immutable image digest, because tags such as latest are mutable. The report therefore anchors the image to its SHA-256 manifest digest, verifies that the Cosign signature refers to that digest, links the SBOM and provenance attestations to the same digest, and separates remediated items from intentionally accepted lab risk.

Evidence Area	Observed Result	Enterprise Meaning
Image digest	sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764	Immutable artifact identity; prevents ambiguity from mutable tags.
Cosign verification	Signature verification succeeded; docker-reference and manifest digest returned by cosign verify.	Provides cryptographic integrity and origin evidence for the pushed GHCR image.
SBOM package count	SPDX packages: 136; CycloneDX components: 2877	Establishes component inventory for dependency, license, and vulnerability review.
Top vulnerable packages	urllib3 (13), requests (6), idna (3), gunicorn (2), flask (1)	Identifies remediation priorities rather than treating all dependencies equally.
Provenance	Builder: private-ubuntu-endpoint-vm; commit: 0478cbcb3a57...	Links artifact back to source revision and local private VM build process.
Accepted risk	Intentional vulnerable packages retained for lab demonstration until secure branch/retest closure.	Shows risk ownership and governance rather than untracked findings.

Enterprise outcome: The evidence package demonstrates artifact integrity, component transparency, vulnerability prioritization, remediation accountability, and an auditable path toward stricter supply-chain gates.

2. Artifact Identity and Image Digest Evidence

Artifact identity is the foundation of supply-chain assurance. In enterprise pipelines, relying only on an image tag is insufficient because tags can be overwritten. The digest is the immutable identifier that should be referenced in verification, promotion records, attestations, deployment approvals, and release evidence.

Field	Value
Container image tag	ghcr.io/jazz00001/enterprise-devsecops:latest
Verified immutable image digest	sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764
Verified docker reference	ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764
Repository	https://github.com/Jazz00001/Enterprise-DevSecOps
Source revision	0478cbcb3a57e9afc07f9b91d9b0e9b7f9ca3b8b
Build type	https://example.com/private-vm/docker-build
Builder identity	private-ubuntu-endpoint-vm
Build start / finish	2026-06-15T13:26:55Z / 2026-06-15T13:26:59Z

Recommended enterprise practice: deployments should prefer a digest-pinned reference such as `image@sha256:<digest>` for production promotion. The tag may still be used for human readability, but policy engines and evidence records should validate the digest.

3. Cosign Image Signing and Verification

Cosign was used to sign the GHCR container image and verify that the artifact has a valid signature. The verification output returns the artifact digest, reference identity, and Cosign signature type. This evidence is stronger than a screenshot alone because it ties the cryptographic verification to a specific immutable digest.

Control	Implementation Evidence	Status
Image signing	Image signed with Cosign against the GHCR artifact.	Completed
Signature verification	cosign verify returned JSON containing docker-manifest-digest and signature type.	Verified
Digest binding	Verification bound to sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764	Verified
Key management note	Lab used Cosign key-pair signing. Production recommendation: use keyless/OIDC or protected KMS/HSM-backed signing.	Improvement noted
Artifact promotion	Signature verification should be required before deployment promotion.	Recommended gate

```
cosign verify --key security/cosign/cosign.pub ghcr.io/jazz00001/enterprise-devsecops:latest
# Result: verified signature bound to digest sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764
```

Verification result interpretation: the signature verification succeeds only when the artifact, digest, and public key verification align. If the pushed image is modified or replaced under the same tag, the digest changes and verification evidence must be regenerated.

4. Syft SBOM Generation and Package Inventory

Syft generated both SPDX and CycloneDX SBOMs for the image. The SPDX document is useful for package inventory and license metadata. CycloneDX adds a supply-chain-focused BOM model that is useful for transparency, dependency risk review, cyber risk reduction, and integration with vulnerability management workflows.

SBOM Artifact	Observed Detail	Enterprise Use
SPDX SBOM	136 package records; created 2026-06-15T13:16:17Z; creator: Organization: Anchore, Inc, Tool: syft-1.45.1	Package inventory, license review, component traceability.
CycloneDX SBOM	2877 components; spec version 1.6; container component digest captured in metadata.	Supply-chain transparency, cyber risk reduction, security tooling integration.
SBOM table report	Human-readable package inventory generated from Syft output.	Reviewer-friendly evidence.
SBOM attachment	SBOM attached/downloaded using Cosign evidence workflow.	Links SBOM to the registry artifact.
SBOM attestation	SBOM attestation verification produced in-toto payload and signature.	Signed claim that SBOM belongs to the artifact.

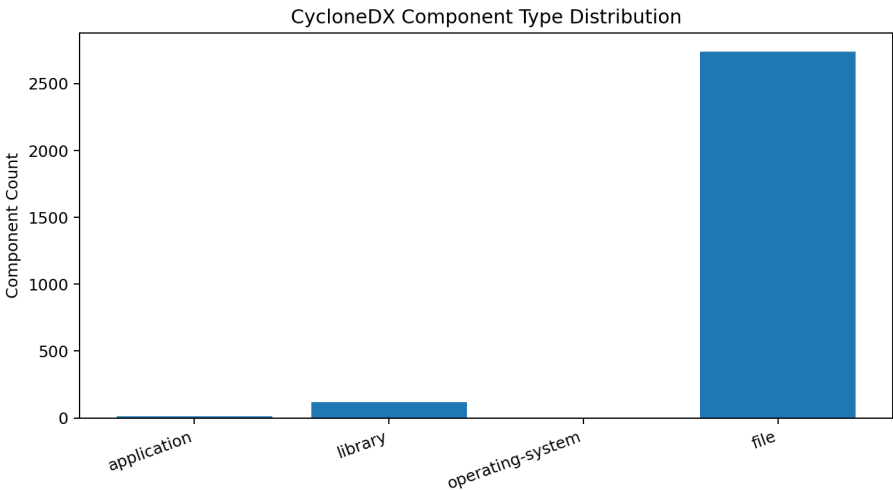


Figure 1 - CycloneDX SBOM component distribution generated from the project SBOM.

Important interpretation: the CycloneDX count is higher than the SPDX package count because CycloneDX captures many file-level components in addition to libraries, applications, and the operating-system component. This is expected behavior when SBOM tooling catalogs a container image deeply.

5. Top Vulnerable Packages and Risk Context

The dependency vulnerability evidence came from pip-audit and Trivy. pip-audit focuses on Python dependency advisories from the application dependency graph, while Trivy adds container image context including OS package findings and Python package findings inside the built image.

Package	Installed Version	Finding Count	Representative IDs	Recommended Fix Direction
urllib3	1.23	13	PYSEC-2019-133, PYSEC-2019-132, PYSEC-2020-148	1.24.2, 1.24.3, 1.25.9, 1.26.17
requests	2.19.1	6	PYSEC-2018-28, PYSEC-2023-74, PYSEC-2023-74	2.20.0, 2.31.0, 2.32.0, 2.32.4
idna	2.7	3	PYSEC-2024-60, PYSEC-2024-60, CVE-2026-45409	3.15, 3.7
gunicorn	21.2.0	2	CVE-2024-1135, CVE-2024-6827	22.0.0
flask	2.3.3	1	CVE-2026-27205	3.1.3

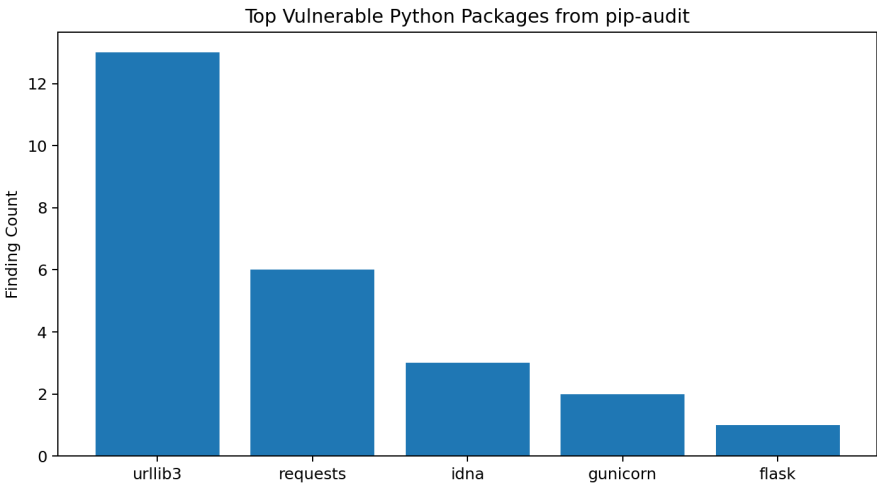


Figure 2 - Top vulnerable Python packages from pip-audit evidence.

The highest priority application dependency in the lab is urllib3 because it is an old transitive dependency introduced by the intentionally old requests version. Requests itself is also a high-priority remediation target because it controls outbound HTTP behavior and pulls transitive HTTP stack dependencies.

6. Why Vulnerable Packages Exist

Not every vulnerability in this project represents accidental negligence. Several vulnerable packages were intentionally introduced or retained to prove that the pipeline can detect, report, prioritize, remediate, and retest vulnerable dependencies. A professional report must clearly distinguish intentional lab exposure from production-acceptable risk.

Source of Vulnerability	Reason It Exists	Security Interpretation
requests==2.19.1	Intentionally old dependency selected to create observable dependency vulnerabilities for pip-audit and Trivy evidence.	Training finding; should be updated in secure branch before production use.
urllib3==1.23	Transitive dependency inherited from old requests version.	Fix should happen by upgrading requests and validating lockfile changes.
idna==2.7	Transitive dependency inherited from old HTTP dependency tree.	Upgrade through dependency refresh and compatibility test.
gunicorn==21.2.0	Pinned runtime server version with known advisories.	Upgrade to fixed version and retest app startup.

Source of Vulnerability	Reason It Exists	Security Interpretation
Flask==2.3.3	Framework version retained during lab construction.	Upgrade after testing route compatibility and session behavior.
Debian base image packages	Base image inherited OS packages and scanner advisories.	Rebuild with updated base image, distroless/Wolfi where appropriate, or documented exception if no fixed package exists.

7. What Was Fixed and What Remains Accepted Risk

The project implemented multiple supply-chain controls even while preserving vulnerable app behavior for demonstration. The goal is not merely to scan; the enterprise-grade goal is to show which risks are fixed, which are deliberately accepted for lab purposes, and which require additional remediation before production promotion.

Area	Completed Fix / Control	Residual Status
Image identity	Image digest captured and used in verification evidence.	Closed for lab evidence.
Image signing	Cosign signature created and verification output stored.	Closed; production should use protected keyless/KMS workflow.
SBOM generation	Syft generated SPDX and CycloneDX SBOMs.	Closed; automate on every build.
SBOM attachment/attestation	SBOM attached to image and attestation verified.	Closed for lab evidence.
Provenance	SLSA-style provenance generated and attested.	Partially closed; full SLSA requires trusted builder provenance.
Application vulnerable dependencies	Detected and documented.	Accepted lab risk until secure-remediation branch upgrades packages.
Container OS vulnerabilities	Detected by Trivy and summarized.	Requires base image refresh or documented exception.

Accepted-risk statement: The vulnerable Flask app intentionally keeps selected vulnerable dependencies and vulnerable code paths for educational attack-and-detect evidence. This is acceptable only in the private lab. Production promotion requires dependency upgrades, passing gates, and updated SBOM/provenance evidence.

Accepted Risk	Business / Lab Rationale	Compensating Control	Expiry / Closure Condition
Old requests/urllib3/idna chain	Required to demonstrate SCA detection and report workflow.	Private lab only; no internet exposure; CI evidence retained.	Close when secure branch updates dependencies and pip-audit passes.
Flask/gunicorn advisories	Shows framework/runtime vulnerability management.	Documented remediation plan and retest commands.	Close when framework/runtime upgrade is tested.
Debian base image findings	Inherited from base image ecosystem during lab build.	Trivy scan artifacts and rebuild plan.	Close after base image refresh or justified exception.
Local SLSA-style provenance	Private VM cannot provide full hosted trusted builder guarantees.	Transparent note in report; Cosign attestation created.	Improve with GitHub Actions artifact attestations or SLSA generator.

8. SLSA and Provenance Alignment

SLSA is relevant because it provides a framework/checklist for preventing tampering, improving software artifact integrity, and securing software supply chains. In this lab, the SLSA-style provenance predicate records the repository, branch, commit, image, digest, builder identity, and build timing.

Provenance Field	Captured Value	Security Value
Repository	https://github.com/Jazz00001/Enterprise-DevSecOps	Links artifact to source control origin.
Reference	refs/heads/main	Identifies branch or release reference.
Commit digest	0478cbcb3a57e9afc07f9b91d9b0e9b7f9ca3b8b	Maps artifact to exact source revision.
Image	ghcr.io/jazz00001/enterprise-devsecops:latest	Links provenance to container artifact.

Provenance Field	Captured Value	Security Value
Image digest	sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764	Binds provenance to immutable artifact.
Builder	private-ubuntu-endpoint-vm	Shows private VM builder identity.
Build timing	2026-06-15T13:26:55Z to 2026-06-15T13:26:59Z	Creates timeline evidence.

Enterprise improvement path: move the build from a local VM to a hardened CI builder, generate signed provenance automatically, use least-privilege tokens, pin GitHub Actions, require branch protection, and verify provenance before deployment.

9. CycloneDX Value for Transparency and Cyber Risk Reduction

CycloneDX is useful because it is a BOM standard designed for software supply-chain transparency and cyber risk reduction. In this project, the CycloneDX SBOM complements SPDX by giving a rich machine-readable representation of components, metadata, types, and artifact relationships.

CycloneDX Element	Project Evidence	Risk-Reduction Use
bomFormat/specVersion	CycloneDX 1.6	Enables tooling to parse a standard BOM.
Metadata component	Container image identified with digest-like version metadata.	Links BOM to artifact identity.
Components	2877 components cataloged	Supports inventory, vulnerability matching, and component ownership.
Component types	application: 15, library: 120, operating-system: 1, file: 2741	Differentiates file, library, application, and OS components.
Operational usage	Can be fed into vulnerability management, exception review, and VEX workflows.	Turns inventory into risk workflow input.

10. Enterprise-Grade CI/CD and Admission Gate Recommendations

To make the Cosign + Syft + SBOM work operationally enterprise-grade, the evidence should be converted into repeatable security gates. The security gates should fail on critical dependency vulnerabilities, critical container vulnerabilities, secrets, critical Dockerfile misconfiguration, and critical Kubernetes manifest misconfiguration.

Gate	Tool / Control	Blocking Rule	Evidence Artifact
Dependency vulnerability	pip-audit / SCA	Critical dependency vulnerability = fail	pip-audit-report.json + summary
Container vulnerability	Trivy image scan	Critical container vulnerability = fail	trivy-image-report.txt/json
Secret detection	Trivy secret scan / Gitleaks	Secret found = fail	secret scan output
Dockerfile config	Trivy config / Hadolint	Dockerfile critical misconfig = fail	config scan report
Kubernetes manifest config	Trivy config / Gatekeeper/Kyverno	Kubernetes manifest critical misconfig = fail	manifest scan + admission evidence
Image signing	Cosign verify	Unsigned or unverifiable image = fail	cosign-image-verify.txt
SBOM generation	Syft	Missing SBOM = fail for release builds	SPDX/CycloneDX SBOM
Provenance	SLSA-style / trusted builder	Missing provenance = warn in lab; fail in production release	slsa-provenance.json + attestation

11. Retest and Evidence Refresh Procedure

Every remediation must produce fresh evidence. The correct retest process is to rebuild the image, push it to the registry, capture the new digest, sign the new digest, regenerate SBOMs, re-run pip-audit and Trivy, re-attest SBOM and provenance, then verify that reports and attestations refer to the same digest.

```
# 1. Rebuild and push image
docker build -t ghcr.io/jazz00001/enterprise-devsecops:latest .
docker push ghcr.io/jazz00001/enterprise-devsecops:latest

# 2. Capture digest
docker buildx imagetools inspect ghcr.io/jazz00001/enterprise-devsecops:latest
```

```
# 3. Sign and verify
cosign sign --key security/cosign/cosign.key ghcr.io/jazz00001/enterprise-devsecops:latest
cosign verify --key security/cosign/cosign.pub ghcr.io/jazz00001/enterprise-devsecops:latest

# 4. Regenerate SBOMs
syft ghcr.io/jazz00001/enterprise-devsecops:latest -o sdx-json=security/sbom/sbom-sdx.json
syft ghcr.io/jazz00001/enterprise-devsecops:latest -o cyclonedx-json=security/sbom/sbom-cyclonedx.json

# 5. Re-run vulnerability checks
pip-audit -r src/app/requirements.txt
trivy image ghcr.io/jazz00001/enterprise-devsecops:latest
```

12. Enterprise Control-to-Evidence Matrix

A supply-chain report becomes enterprise-grade when every claim has a corresponding evidence artifact and a clear operational decision. The matrix below connects controls to their technical proof, release decision, and audit value.

Control Objective	Evidence Artifact	Release Decision	Audit Value
Immutable artifact identity	cosign-image-verify.txt and provenance imageDigest	Required for release candidate approval	Proves the exact artifact under review.
Signature verification	Cosign verify JSON output	Fail if signature is missing or invalid	Confirms integrity and signer/key relationship.
SBOM generation	sbom-sdx.json and sbom-cyclonedx.json	Fail if SBOM is missing in release build	Creates component transparency and SCA input.
SBOM package count	SPDX package count and CycloneDX component count	Warn if count changes unexpectedly	Detects unexpected dependency drift.
Top vulnerable packages	pip-audit report and Trivy image report	Fail on critical production findings	Turns scan output into prioritized remediation.
Provenance attestation	slsa-provenance.json and verification output	Warn in lab; fail in production once trusted builder is enabled	Links artifact to source, commit, builder and build metadata.
Accepted-risk governance	Exception table and VEX notes	Fail if exception has no owner, reason, expiry, or compensating control	Prevents silent risk accumulation.
Retest completeness	Fresh scans after remediation	Required before closure	Proves remediation changed the result.

13. Package-Level Remediation Backlog

The dependency backlog below translates scanner output into actionable engineering tasks. This is the type of section recruiters and security reviewers expect because it proves that findings are understood, prioritized, and connected to concrete fixes.

Package / Layer	Issue Source	Enterprise Fix	Validation Step	Risk Status
requests 2.19.1	pip-audit and Trivy Python package scan	Upgrade requests to a fixed supported version and regenerate lockfile or pinned requirements.	Run pip-audit and functional HTTP regression tests.	Accepted lab risk until remediation branch.
urllib3 1.23	Transitive dependency from old requests	Upgrade through requests refresh or explicitly constrain urllib3 to fixed compatible version.	Confirm no urllib3 advisories remain in pip-audit.	High-priority remediation.
idna 2.7	Transitive HTTP dependency	Refresh dependency tree and validate URL/IDNA handling tests.	Run pip-audit and application route tests.	Medium/high priority depending on advisory.
gunicorn 21.2.0	Runtime server advisory	Upgrade gunicorn to fixed release and verify container startup/readiness probes.	Run Docker healthcheck and Kubernetes readiness probe.	Should be fixed before production.
Flask 2.3.3	Framework advisory	Upgrade Flask and validate route behavior, template rendering, and session/cache behavior.	Run pytest, Bandit, and manual curl smoke tests.	Documented remediation.
Debian base layer	Trivy OS package scan	Rebuild from patched base image or move to reduced/distroless base where compatible.	Re-run Trivy image scan and compare delta.	Accepted only with expiry and rebuild plan.

14. VEX-Style Decision Notes

VEX-style notes explain whether a vulnerability is exploitable, fixed, not affected, under investigation, or accepted for a defined reason. In this lab, the project should not simply list CVEs; it should document whether the dependency is intentionally present, whether the vulnerable path is reachable, and when the risk must be closed.

Decision Category	When to Use	Project Example	Required Evidence
Affected - fix planned	The vulnerable package is present and reachable or cannot be ruled out.	requests/urllib3/idna chain in the private lab image.	pip-audit report, upgrade task, retest result.
Under investigation	The package is present but exploitability needs deeper validation.	Debian OS advisories with no immediate application exploit path confirmed.	Trivy report, package version, vendor status.
Not affected	The vulnerable code path is not present or not reachable.	Future advisory triage where vulnerable feature is unused.	Code review note, runtime configuration, test proof.
Fixed	The version was upgraded and scanners no longer flag the issue.	Secure branch after dependency upgrades.	Before/after reports and passing CI gates.
Accepted risk	Risk remains temporarily for a business or lab reason.	Intentionally vulnerable packages retained for demonstration.	Owner, reason, compensating control, expiry date, no public exposure.

Governance rule: No accepted risk should remain open indefinitely. Every accepted risk requires an owner, expiry condition, compensating control, and retest evidence when closed.

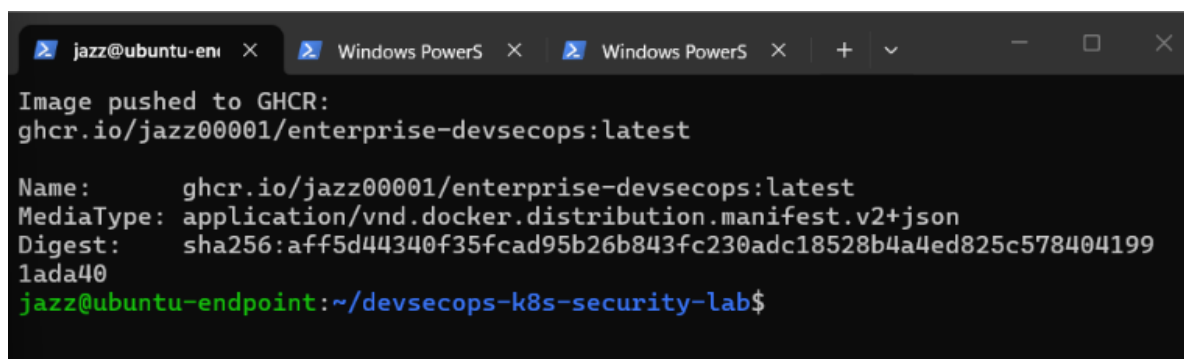
15. Production Maturity Roadmap

The current project is strong for a private lab. To make the same design production-grade, the controls should be moved from manual evidence collection to automated, enforced, and continuously monitored supply-chain gates.

Maturity Step	Current Lab State	Enterprise Target
Signing model	Cosign key-pair signing in private lab	Keyless signing with OIDC or KMS/HSM-backed signing.
Provenance	SLSA-style local provenance predicate	Trusted CI-generated provenance with protected build identity.
SBOM generation	Manual Syft generation and attachment	Automatic SBOM generation on every build and release.
SBOM format	SPDX and CycloneDX generated	Store both; use CycloneDX for risk/VEX workflows and SPDX for license/package inventory.
Deployment verification	Evidence documented in reports	Admission controller verifies signature, digest, labels and provenance before deploy.
Exception workflow	Accepted-risk sections in PDF	Tracked exceptions in repository with owner, expiry and review cadence.
Scorecard posture	OpenSSF Scorecard recommended	Run Scorecard on schedule and enforce repository posture improvements.

16. Selected Evidence Screenshots

The screenshots below support the supply-chain evidence narrative. They are selected from the project evidence set and should be stored in the repository evidence folder alongside machine-readable reports.



```

jazz@ubuntu-eni x Windows PowerS x Windows PowerS x + v - □ x
Image pushed to GHCR:
ghcr.io/jazz00001/enterprise-devsecops:latest

Name:      ghcr.io/jazz00001/enterprise-devsecops:latest
MediaType: application/vnd.docker.distribution.manifest.v2+json
Digest:    sha256:aff5d44340f35fcad95b26b843fc230adc18528b4a4ed825c578404199
1ada40
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

GHCR image pushed - registry artifact available for signing and scanning.


```
jazz@ubuntu-eni x Windows PowerShell x Windows PowerShell x + - □ x
Current image digest:
ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95a6aab7a76d5715613
d359a0e92be22716b44c1f7514bc720764

Signing current digest:
Enter password for private key:
Signing artifact... \ Pushing signature to: ghcr.io/jazz00001/enterprise-dev
secops

Verifying Cosign image signature:

Verification for ghcr.io/jazz00001/enterprise-devsecops@sha256:14776e0fe1b95
a6aab7a76d5715613d359a0e92be22716b44c1f7514bc720764 --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The signatures were verified against the specified public key

[{"critical":{"identity":{"docker-reference":"ghcr.io/jazz00001/enterprise-d
evsecops@sha256:14776e0fe1b95a6aab7a76d5715613d359a0e92be22716b44c1f7514bc72
0764"},"image":{"docker-manifest-digest":"sha256:14776e0fe1b95a6aab7a76d5715
613d359a0e92be22716b44c1f7514bc720764"},"type":"https://sigstore.dev/cosign/
sign/v1"},"optional":{}}]
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Cosign verification evidence - signature bound to image digest.

```
jazz@ubuntu-eni x Windows PowerShell x Windows PowerShell x + - □ x
SBOM files:
total 3.5M
-rw-r--r-- 1 jazz jazz 1.1M Jun 15 13:17 sbom-cyclonedx.json
-rw-r--r-- 1 jazz jazz 2.4M Jun 15 13:16 sbom-spdx.json

SPDX package count:
136

First few packages:
{
  "name": "Simple Launcher",
  "version": "1.1.0.14"
}
{
  "name": "Simple Launcher",
  "version": "1.1.0.14"
}
{
  "name": "Simple Launcher",
  "version": "1.1.0.14"
}
{
  "name": "Simple Launcher",
  "version": "1.1.0.14"
}
{
  "name": "Simple Launcher",
  "version": "1.1.0.14"
}
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$ |
```

Syft SBOM generation evidence - package inventory produced.

```
jazz@ubuntu-eni x Windows PowerShell x Windows PowerShell x + v - □ ×
```

```
Attaching SBOM to image:  
WARNING: SBOM attachments are deprecated and support will be removed in a Co  
sign release soon after 2024-02-22 (see https://github.com/sigstore/cosign/i  
ssues/2755). Instead, please use SBOM attestations.  
WARNING: Attaching SBOMs this way does not sign them. To sign them, use 'cos  
ign attest --predicate security/sbom/sbom-spdx.json --key <key path>'.  
Uploading SBOM file for [ghcr.io/jazz00001/enterprise-devsecops:latest] to [  
ghcr.io/jazz00001/enterprise-devsecops:sha256-14777e0fe1b95a6aab7a76d5715613  
d359a0e92be22716b44c1f7514bc728764.sbm] with mediaType [text/spdx+json].  
  
Downloading attached SBOM back from registry:  
WARNING: SBOM attachments are deprecated and support will be removed in a Co  
sign release soon after 2024-02-22 (see https://github.com/sigstore/cosign/i  
ssues/2755). Instead, please use SBOM attestations.  
WARNING: Downloading SBOMs this way does not ensure its authenticity. If you  
want to ensure a tamper-proof SBOM, download it using 'cosign download atte  
station <image uri>'.  
Found SBOM of media type: text/spdx+json  
  
Downloaded SBOM:  
-rw-rw-r-- 1 jazz jazz 2.4M Jun 15 13:22 security/reports/downloaded-sbom-sp  
dx.json  
  
Downloaded SBOM package count:  
136  
jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

SBOM attached and downloaded from registry evidence.

[illegible]

SBOM attestation verification evidence.

SLSA-style provenance predicate created.

Provenance attestation verified.

17. References and Standards Alignment

The report aligns the project evidence to recognized supply-chain security practices and formats. The references below are included to support the terminology and rationale used in the report.

Reference	Relevance	URL
SLSA	Supply-chain Levels for Software Artifacts - security framework/checklist for preventing tampering, improving integrity, and securing packages and infrastructure.	https://slsa.dev/
CycloneDX	OWASP CycloneDX - full-stack Bill of Materials standard with supply-chain capabilities for cyber risk reduction.	https://cyclonedx.org/
Sigstore Cosign	Cosign signing and verification for container images and supply-chain attestations.	https://docs.sigstore.dev/cosign/
Syft	Syft CLI and library for generating SBOMs from container images and filesystems.	https://github.com/anchore/syft

Final statement: The project demonstrates enterprise-grade supply-chain evidence by linking the container artifact to its digest, signature, SBOM, vulnerability context, provenance, and risk decisions. The remaining work is to convert these controls into mandatory release gates and refresh evidence after every build.